Map, Set, WeakMap & WeakSet

10 min read · Updated 17 Jul 2026 · Topic 20 of 23 · Bookmark · Completed

`Map`, `Set`, `WeakMap`, and `WeakSet` are the keyed collections ES2015 added to replace the two things people used to abuse plain objects for: a lookup table and a "have I seen this?" flag. Interviewers reach for them because they expose whether you understand *reference identity*, *iteration order*, and *garbage collection* — the same three ideas show up in caching, deduping, and memory-leak questions. This lesson covers each collection, when to pick it, how to convert to and from arrays/objects, and the traps.

Map — a real key/value collection

A `Map` holds key/value pairs where the key can be **any value**, including objects, functions, and `NaN` — not just strings. It preserves **insertion order**, exposes a `.size`, and is directly iterable.

JS Copy

```
const m = new Map();
const objKey = { id: 1 };
m.set('name', 'Ada');
m.set(1, 'number one');
m.set(objKey, 'value for object');
m.set(NaN, 'not a number');
console.log(m.get('name'));   // ⇒ Ada
console.log(m.get(objKey));   // ⇒ value for object
console.log(m.get(NaN));      // ⇒ not a number
console.log(m.size);          // ⇒ 4
console.log(m.has(1));        // ⇒ true
console.log([...m.keys()]);   // ⇒ [ 'name', 1, { id: 1 }, NaN ]
```

though `NaN !== NaN`; Map compares keys with the **SameValueZero** algorithm, under which `NaN` equals `NaN` (and `+0` equals `-0`).

Map iterates in insertion order via `for ... of`, `.forEach()`, `.keys()`, `.values()`, and `.entries()`. Note `.forEach` passes `(value, key)` — value first, mirroring arrays.

JS Copy

```
const scores = new Map([
  ['bob', 42],
  ['ana', 91],
  ['cy', 68],
]);
for (const [name, score] of scores) console.log(name, score);
// ⇒ bob 42 / ana 91 / cy 68  (insertion order, guaranteed)
scores.forEach((score, name) ⇒ console.log(name, '⇒', score));
// ⇒ bob ⇒ 42 / ana ⇒ 91 / cy ⇒ 68
```

Map vs plain object

The interview classic. A plain object *looks* like a map, but it is a fundamentally different tool. Reach for `Map` whenever keys are dynamic, non-string, or numerous, or when you insert/delete a lot.

Concern	`Map`	Plain object `{}`
Key types	any value (objects, functions, `NaN`, numbers)	strings & symbols only — eve
Order	guaranteed insertion order	mostly insertion order, but **in**
Size	`map.size` (O(1))	no built-in; `Object.keys(o)`
Iteration	directly iterable (`for ... of`, spread)	not iterable; need `Object.k`
Prototype pollution	none — a Map has no default keys	inherits `toString`, `constr`

Performance	optimized for frequent add/delete	fine for fixed-shape records;
Serialization	not JSON-native (convert first)	`JSON.stringify` works dire

The prototype-pollution row is the one juniors miss: with a plain object used as a dictionary, a user-supplied key like `"__proto__"` or `"hasOwnProperty"` can shadow inherited members or corrupt the prototype. `Map` has no such surface — its keys never touch a prototype chain. Use `Object.create(null)` if you must keep an object dictionary.

Set — a collection of unique values

A `Set` stores **unique** values (again by SameValueZero), in insertion order, iterable, with a `.size`. Its headline use is deduping and O(1)-ish membership tests.

JS Copy

```
const s = new Set([1, 2, 2, 3, 3, 3]);
console.log(s.size);   // ⇒ 3
console.log([...s]);   // ⇒ [ 1, 2, 3 ]

const dupes = ['a', 'b', 'a', 'c', 'b'];
const unique = [...new Set(dupes)];
console.log(unique);   // ⇒ [ 'a', 'b', 'c' ]

console.log(s.has(2)); // ⇒ true  (fast membership, vs array.includes O(n
console.log(new Set([NaN, NaN]).size); // ⇒ 1  (NaN dedups here)
```

`[...new Set(arr)]` is the idiomatic one-liner for "dedup an array while keeping order". `Set.prototype.has` is the reason to use a Set as a "seen" guard inside a loop instead of `array.includes`, which rescans from the start each time.

Set operations (union / intersection / difference)

Interviewers often ask you to implement these from primitives, which tests spread + `filter` + `has`.

JS Copy

```
const a = new Set([1, 2, 3, 4]);
const b = new Set([3, 4, 5, 6]);
const union        = new Set([...a, ...b]);
const intersection = new Set([...a].filter(x => b.has(x)));
const difference   = new Set([...a].filter(x => !b.has(x)));
console.log([...union]);        // => [ 1, 2, 3, 4, 5, 6 ]
console.log([...intersection]); // => [ 3, 4 ]
console.log([...difference]);   // => [ 1, 2 ]
```

Modern engines (Node 22+, current browsers, ES2024) ship these as **native methods**, which are cleaner and lazier — good to name-drop, but be ready to hand-roll the versions above if asked:

JS Copy

```
const a = new Set([1, 2, 3, 4]);
const b = new Set([3, 4, 5, 6]);
console.log([...a.union(b)]);        // => [ 1, 2, 3, 4, 5, 6 ]
console.log([...a.intersection(b)]); // => [ 3, 4 ]
console.log([...a.difference(b)]);   // => [ 1, 2 ]
console.log(a.isSubsetOf(new Set([1, 2, 3, 4, 5]))); // => true
```

WeakMap & WeakSet — references that don't keep objects alive

`WeakMap` and `WeakSet` look like stripped-down versions of `Map` / `Set`, and the constraints are the whole point:

- **Keys (WeakMap) / values (WeakSet) must be objects** — primitives throw a `TypeError`.
- References are held **weakly**: if nothing else in the program references the key object, the garbage collector can reclaim it *and* drop the entry. A regular `Map` key, by contrast, keeps its object alive forever — a classic leak.

iterate. The API is only `get` / `set` / `has` / `delete` (WeakMap) or `add` / `has` / `delete` (WeakSet).

JS Copy

```
const wm = new WeakMap();
let key = { big: 'data' };
wm.set(key, 'metadata');
console.log(wm.has(key)); // ⇒ true
key = null;               // last strong reference dropped
// The { big:'data' } object is now unreachable, so GC may reclaim it
// AND remove the WeakMap entry automatically. No leak, no manual cleanup.

new WeakMap().set('str', 1); // ⇒ throws TypeError: Invalid value used as
```

Use case 1 — private data

Before class private fields (`#field`), the canonical way to attach truly private state to an instance was a module-scoped `WeakMap` keyed by `this`. Nothing leaks onto the instance, and when the instance is collected so is its private record.

JS Copy

```
const _private = new WeakMap();
class Account {
  constructor(balance) { _private.set(this, { balance }); }
  deposit(n) { _private.get(this).balance += n; return this; }
  get balance() { return _private.get(this).balance; }
}
const acc = new Account(100);
acc.deposit(50);
console.log(acc.balance);          // ⇒ 150
console.log(Object.keys(acc));     // ⇒ []   (nothing enumerable on the in
console.log(JSON.stringify(acc)); // ⇒ {}   (the balance is invisible/pri
```

Use case 2 — caching / memoization tied to object lifetime

must manually evict.

JS Copy

```
const cache = new WeakMap();
function heavyCompute(obj) {
  if (cache.has(obj)) return cache.get(obj);
  const result = Object.keys(obj).length * 1000; // pretend-expensive
  cache.set(obj, result);
  return result;
}
const config = { a: 1, b: 2 };
console.log(heavyCompute(config)); // ⇒ 2000  (computed)
console.log(heavyCompute(config)); // ⇒ 2000  (served from cache)
```

Use case 3 — metadata / "have I processed this?" without leaks

A `WeakSet` tags objects (DOM nodes, records) as "seen / visited / initialized" without preventing them from being garbage-collected.

JS Copy

```
const initialized = new WeakSet();
function setup(node) {
  if (initialized.has(node)) return; // idempotent
  initialized.add(node);
  // ...wire up the node once...
}
```

Choosing the right collection

- **Ordered key/value where keys aren't all strings, or where you add/delete a lot →** `Map`.
- **Fixed-shape record / needs JSON serialization / small and known keys →** plain object.
- **Unique values, deduping, fast membership →** `Set`.

If you need to iterate, count, or serialize, you need the strong (non-Weak) variant — the weak ones deliberately can't do any of those.

Converting between collections and arrays/objects

JS Copy

```
const scores = new Map([['bob', 42], ['ana', 91]]);
console.log([...scores]);                 // ⇒ [ [ 'bob', 42 ], [ 'ana', 91
console.log(Object.fromEntries(scores));// ⇒ { bob: 42, ana: 91 }
console.log(new Map(Object.entries({ a: 1 }))); // object → Map
console.log([...new Set([1, 1, 2])]);    // ⇒ [ 1, 2 ]
console.log(new Set([1, 2, 3]));          // array → Set
```

`Object.fromEntries` only round-trips cleanly when every key is a string/symbol — a Map with object or numeric keys loses information when flattened to an object.

Common interview questions

Q: When would you use a `Map` over a plain object? When keys are non-strings (especially objects) or dynamic/user-supplied, when you need guaranteed insertion order regardless of key shape, when you insert and delete frequently, or when you want to avoid prototype-pollution and inherited-key collisions. Use a plain object for small fixed records and anything you must `JSON.stringify` directly.

Q: Why use a `WeakMap` for private data instead of a `Map`? A `Map` keyed by `this` holds a **strong** reference to every instance, so instances can never be garbage-collected as long as the map lives — a memory leak. A `WeakMap` holds the key weakly, so when the instance becomes unreachable elsewhere, both it and its private record are collected automatically.

Q: How do you deduplicate an array? `[...new Set(arr)]` — Set drops duplicates by SameValueZero and spread turns it back into an array, preserving first-seen order. (Caveat: this dedupes primitives and *object identity*, not deep-equal objects.)

Q: Why can't you iterate a `WeakMap`? Because entries can be removed by the garbage collector at any moment, there is no deterministic set of contents to enumerate; exposing iteration would leak GC timing into observable program behavior. Hence no iterator, no `.size`, no `.clear()`.

Gotchas / traps

Object keys are compared by reference, not structure. Two objects with identical contents are different keys.

JS Copy

```
const m = new Map();
m.set({ x: 1 }, 'a');
console.log(m.get({ x: 1 })); // ⇒ undefined  (different object literal)
const k = { x: 1 };
m.set(k, 'b');
console.log(m.get(k));        // ⇒ b          (same reference)
```

Plain-object keys collapse under string coercion; Map keys don't. This is why a Map is safer for mixed-type keys.

JS Copy

```
const o = {};
o[1] = 'num';
o['1'] = 'str';
console.log(o[1]);            // ⇒ str        (1 and '1' are the same key
console.log(Object.keys(o));  // ⇒ [ '1' ]
const m = new Map().set(1, 'num').set('1', 'str');
console.log(m.size);          // ⇒ 2          (kept distinct)
```

`NaN` **behaves *unusually well* here.** Everywhere else `NaN !== NaN`, but as a Map key or Set value it matches itself, so `new Set([NaN, NaN]).size === 1` and `map.get(NaN)` works. Interviewers use this to check you know SameValueZero.

JS

Copy

```
const wm = new WeakMap([[{}, 1]]);
console.log('size' in wm);                // ⇒ false
console.log(typeof wm[Symbol.iterator]); // ⇒ undefined  (not iterable)
// for (const _ of wm) {}  // ⇒ TypeError: wm is not iterable
```

A `WeakMap` key must be an object. `wm.set('id', …)` or `wm.set(42, …)` throws `TypeError`. (Recent engines also allow registered symbols as weak keys, but plain primitives never work.)

`Map` / `Set` don't serialize to JSON. `JSON.stringify(new Map([['a',1]]))` yields `"{}"`, silently dropping your data — convert with `Object.fromEntries` / `[…set]` first.

QUICK CHECK — no marks, just for you.

1. What does this snippet log?

JAVASCRIPT

Copy

```
const arr = [1, 2, 2, 3, 3, 3];
const unique = [...new Set(arr)];
console.log(unique);
console.log(new Set([NaN, NaN]).size);
```

- ○ `[ 1, 2, 3, 3 ]` `2`
- ○ `[ 1, 1, 2, 3 ]` `1`
- ○ `[ 1, 2, 3 ]` `1`
- ○ `[ 1, 2, 3 ]` `2`

2. What does this snippet log?

JAVASCRIPT Copy

```
const m = new Map();
m.set({ x: 1 }, 'a');
m.set(NaN, 'nan');
console.log(m.get({ x: 1 }));
console.log(m.get(NaN));
console.log(m.size);
```

- undefined
  nan
  2
- a
  nan
  2
- undefined
  undefined
  2
- a
  undefined
  1

Check answer

3. Why would you use a WeakMap keyed by `this` for an instance's private data instead of a regular Map?

- A WeakMap can store primitive keys, so it can key on the instance's id instead of the object.

A WeakMap is iterable and exposes .size, making it easier to audit stored private records.

- A Map cannot use objects as keys, so only a WeakMap can key by `this`.
- A Map holds a strong reference to every instance key, preventing garbage collection, while a WeakMap holds keys weakly so the instance and its private record are collected once the instance is unreachable.

Check answer

‹ Previous
Destructuring, Spread & Rest

Next ›
Modules: ESM vs CommonJS

LEARN MORE

ARTICLES & DOCS

- Map (MDN)
  MDN
- WeakMap (MDN)
  MDN

VIDEOS



- Map, Set, WeakMap & WeakSet (interview)
  YouTube



- Map/Set/WeakMap/WeakSet — real use case
  YouTube



- Map vs WeakMap
  YouTube

Hand-picked references for this lesson.

JavaScript Interview Kit